

Dynamic Graphs, HLD y Dynamic Trees para el Final de EDA

Guía desde cero, orientada a resolver problemas teóricos

Como usar esta guía

Esta guía está basada en los PDFs de la carpeta `dinamico/`. Está escrita como si no supieras nada del tema: primero define el vocabulario, luego explica la estructura, luego muestra como reconocer problemas y como justificar correctitud/complejidad en papel.

PDFs cubiertos:

- `CS3014___Heavy_Light_Decomposition.pdf`: HLD, path queries, path updates, subtree updates y small-to-large trick.
- `eda_slides_08_dynamic_graph.pdf`: dynamic tree problem, preferred paths, auxiliary splay trees, link-cut trees y análisis con HLD.
- `eda_slides_09_dynamic_graph.pdf`: dynamic connectivity, Euler-tour trees, conectividad en grafos generales y lower bounds.

El foco del examen no es memorizar código. El foco es poder decir: que problema tengo, que estructura aplica, que invariantes mantiene, como responde queries, por que es correcto y cuanto cuesta.

Contents

1	Mapa mental para clasificar problemas dinámicos	4
1.1	Primera pregunta: que cambia?	4
1.2	Segunda pregunta: el grafo siempre es bosque?	4
1.3	Tercera pregunta: pide conectividad o agregados?	4
1.4	Checklist de respuesta de examen	4
2	Diccionario fundamental desde cero	5
2.1	Grafo dinámico	5
2.2	Conectividad	5
2.3	Componente conexa	5
2.4	Bosque	5
2.5	Arista de árbol y arista no-árbol	5
2.6	Online vs offline	5
2.7	Amortizado	5
2.8	Agregado	5
2.9	Segment tree	6
2.10	Treap, AVL, splay y BST balanceado	6
2.11	Splay tree	6
3	HLD: Heavy-Light Decomposition	7
3.1	Que problema resuelve	7
3.2	Idea principal	7
3.3	Subtree size	7
3.4	Arista pesada y arista liviana	7
3.5	Lema clave	7

3.6	Por que el lema implica $O(\log n)$	8
3.7	Arreglos que guarda HLD	8
3.8	Propiedad de linealizacion	8
3.9	Consulta de camino con HLD	8
3.10	Correctitud de HLD path query	8
3.11	Complejidad	9
3.12	Subtree queries con el mismo HLD	9
3.13	Nodos vs aristas	9
3.14	Cuando usar HLD	9
3.15	Cuando NO usar HLD	9
4	Small-to-large trick / DSU on tree	10
4.1	Que problema resuelve	10
4.2	Idea intuitiva	10
4.3	Relacion con HLD	10
4.4	Pseudocodigo teorico	10
4.5	Correctitud	10
4.6	Complejidad	10
5	Dynamic tree problem	11
5.1	Modelo	11
5.2	Diferencia con HLD	11
5.3	Operaciones tipicas	11
6	Link-cut trees	12
6.1	Idea central	12
6.2	Preferred child	12
6.3	Preferred edge y preferred path	12
6.4	Auxiliary tree	12
6.5	Path-parent	12
6.6	La operacion $\text{access}(v)$	12
6.7	Por que access permite path queries	12
6.8	Analisis con HLD	13
6.9	Correctitud conceptual	13
6.10	Cuando usar link-cut tree	13
6.11	Cuando NO usar link-cut tree	13
7	Euler-tour trees	14
7.1	Idea central	14
7.2	Secuencia Euler	14
7.3	Representacion en BST balanceado	14
7.4	Cut en Euler-tour tree	14
7.5	Link en Euler-tour tree	14
7.6	Conectividad con Euler-tour trees	14
7.7	Comparacion con link-cut trees	15
8	Dynamic connectivity en grafos generales	16
8.1	Problema	16
8.2	Por que es mas dificil que bosques	16
8.3	Estrategia de los slides	16
8.4	Idea de niveles	16
8.5	Lower bounds	16

9	Rollback DSU y conectividad dinamica offline	17
9.1	Por que incluirlo	17
9.2	DSU normal	17
9.3	Rollback DSU	17
9.4	Segment tree sobre el tiempo	17
9.5	Correctitud	17
9.6	Complejidad	17
9.7	Component sum con vertex add	18
9.8	Cuando usar rollback DSU	18
10	Imperial Roads y max edge on path	19
10.1	Conexion con dynamic/trees	19
10.2	Propiedad del ciclo	19
10.3	Estructura	19
10.4	Correctitud	19
11	Como reconocer que estructura usar	20
12	Plantillas de solucion teorica	20
12.1	Plantilla HLD	20
12.2	Plantilla link-cut tree	20
12.3	Plantilla Euler-tour tree	20
12.4	Plantilla rollback DSU	21
13	Mapa de slides cubiertos	22
14	Apendice: mapa pagina por pagina de los slides dinamicos	23

1 Mapa mental para clasificar problemas dinamicos

1.1 Primera pregunta: que cambia?

Un problema dinamico es uno donde la estructura cambia en el tiempo. Antes de elegir un algoritmo, identifica que tipo de cambio existe:

Tipo	Que operaciones aparecen	Estructuras candidatas
Incremental	solo inserta aristas	DSU
Decremental	solo borra aristas	DSU offline en reversa, estructuras decrementales
Fully dynamic	inserta y borra aristas	Euler-tour trees, link-cut trees, rollback DSU offline
Arbol dinamico	link/cut entre arboles	Link-cut tree, Euler-tour tree
Arbol estatico con path updates	no cambia la forma, cambian valores	HLD + segment tree
Subarboles en arbol estatico	consultas por subarbol	Euler tour timestamps, DSU on tree

1.2 Segunda pregunta: el grafo siempre es bosque?

Esta pregunta cambia todo:

- Si siempre es un bosque, cada componente es un arbol. Operaciones `link` y `cut` son naturales.
- Si es un grafo general, puede haber ciclos. Al borrar una arista del spanning forest, debes buscar una arista no-arbol que reconecte la componente.
- Si el problema es offline, puedes evitar estructuras complicadas usando segment tree sobre el tiempo + rollback DSU.

1.3 Tercera pregunta: pide conectividad o agregados?

- Solo conectividad incremental: DSU.
- Conectividad fully dynamic online: Euler-tour trees por niveles o link-cut trees si el grafo se mantiene como bosque.
- Agregado en camino de arbol estatico: HLD + segment tree.
- Agregado en camino de arbol dinamico: link-cut tree.
- Agregado en componente con updates offline: rollback DSU con suma por componente.

1.4 Checklist de respuesta de examen

Para cualquier problema de esta carpeta escribe:

1. **Modelo:** arbol estatico, bosque dinamico o grafo general.
2. **Operaciones:** link, cut, insert, delete, connected, query path, query subtree, update path, update vertex.
3. **Estructura:** HLD, DSU, rollback DSU, Euler-tour tree, link-cut tree.
4. **Invariante:** que representa cada arreglo/arbol auxiliar.
5. **Operacion:** como se transforma una query en rangos o en splits.
6. **Correctitud:** por que no pierdes aristas/nodos relevantes.
7. **Complejidad:** preproceso, update, query y espacio.

2 Diccionario fundamental desde cero

2.1 Grafo dinamico

Un grafo dinamico es un grafo que cambia. En vez de recibir un grafo fijo y responder una sola pregunta, recibes una secuencia de operaciones:

$$op_1, op_2, \dots, op_q.$$

Cada operacion puede insertar una arista, borrar una arista, preguntar si dos vertices estan conectados o pedir algun agregado.

2.2 Conectividad

Dos vertices u y v estan conectados si existe un camino entre ellos. Una consulta `connected(u,v)` pregunta si u y v pertenecen a la misma componente conexa.

2.3 Componente conexa

Una componente conexa es un conjunto maximo de vertices donde todos estan conectados entre si. “Maximo” significa que no puedes agregar otro vertice sin romper la propiedad.

2.4 Bosque

Un bosque es un grafo sin ciclos. Cada componente conexa del bosque es un arbol. Muchas estructuras dinamicas primero resuelven bosques porque link/cut son mas faciles ahi.

2.5 Arista de arbol y arista no-arbol

En un grafo general puedes mantener un spanning forest. Las aristas dentro de ese bosque son **tree edges**. Las otras aristas son **non-tree edges**. Si borras una tree edge, la componente puede partirse y necesitas buscar una non-tree edge que vuelva a conectar ambas partes.

2.6 Online vs offline

Un algoritmo **online** responde cada operacion antes de ver las futuras. Un algoritmo **offline** conoce toda la secuencia de operaciones desde el inicio. Esto importa muchisimo:

- Online fully dynamic connectivity es dificil.
- Offline fully dynamic connectivity se puede resolver con segment tree sobre el tiempo + rollback DSU.

2.7 Amortizado

Una complejidad amortizada no garantiza que cada operacion individual cueste poco, sino que el promedio sobre una secuencia larga cuesta poco. Link-cut trees tienen operaciones $O(\log n)$ amortizadas: algunas pueden ser mas caras, pero el total de muchas operaciones queda acotado.

2.8 Agregado

Un agregado es un valor combinado sobre un conjunto:

$$\sum, \min, \max, \gcd, \text{ xor}.$$

Ejemplos:

- suma de valores de una componente;
- maximo peso en un camino;
- minimo peso en el camino entre dos nodos;
- suma de un subarbol.

2.9 Segment tree

Un segment tree mantiene informacion de intervalos de un arreglo. En esta carpeta aparece como complemento de HLD: HLD convierte caminos de arbol en pocos intervalos de un arreglo, y el segment tree responde esos intervalos.

2.10 Treap, AVL, splay y BST balanceado

Un BST balanceado representa una secuencia con operaciones:

split, merge, insert, erase, aggregate.

Euler-tour trees guardan el tour de un arbol en un BST balanceado. Link-cut trees guardan preferred paths en splay trees.

2.11 Splay tree

Un splay tree es un BST autoajutable. Su operacion clave es **splay**(x): mueve x a la raiz mediante rotaciones. Su garantia es amortizada. Link-cut trees usan splay trees para representar caminos auxiliares.

3 HLD: Heavy-Light Decomposition

3.1 Que problema resuelve

HLD resuelve consultas y modificaciones sobre caminos en un **arbol estatico**. El arbol no cambia de forma. Lo que puede cambiar son valores en nodos o aristas.

Ejemplos:

- maximo valor en el camino $u \rightsquigarrow v$;
- sumar w a todos los nodos del camino $u \rightsquigarrow v$;
- consultar suma en el camino;
- actualizar una arista;
- consultar un subarbol usando timestamps.

3.2 Idea principal

Un camino arbitrario en un arbol puede ser largo y curvo. HLD parte el arbol en cadenas pesadas. La magia es que cualquier camino raiz-nodo cruza solo $O(\log n)$ aristas livianas, por tanto cualquier camino entre dos nodos se descompone en $O(\log n)$ segmentos de cadenas.

Luego, cada cadena se coloca en posiciones consecutivas de un arreglo. Una query de camino se vuelve varias queries de intervalo.

3.3 Subtree size

Para cada nodo:

$$\text{size}(u) = \text{numero de nodos en el subarbol de } u.$$

Se calcula con DFS:

$$\text{size}(u) = 1 + \sum_{v \in \text{children}(u)} \text{size}(v).$$

3.4 Arista pesada y arista liviana

Hay dos definiciones en los slides.

Definicion 1: mayor que la mitad

La arista (u, v) es pesada si:

$$2 \text{size}(v) > \text{size}(u).$$

Las demas son livianas.

Definicion 2: mayor que sus hermanos

La arista (u, v) es pesada si v es el hijo de mayor subarbol entre todos los hijos de u . Si hay empate, eliges solo uno. Las demas son livianas.

3.5 Lema clave

Al bajar por una arista liviana (u, v) :

$$\text{size}(v) \leq \frac{\text{size}(u)}{2}.$$

3.6 Por que el lema implica $O(\log n)$

Cada vez que bajas por una arista liviana, el tamaño del subarbol se reduce al menos a la mitad:

$$n, \quad \frac{n}{2}, \quad \frac{n}{4}, \quad \frac{n}{8}, \dots$$

No puedes dividir entre dos mas de $\log_2 n$ veces antes de llegar a 1. Por eso en cualquier camino desde la raíz hasta un nodo hay a lo mucho $O(\log n)$ aristas livianas. Entre aristas livianas hay cadenas pesadas, así que también hay $O(\log n)$ cadenas pesadas.

3.7 Arreglos que guarda HLD

Los slides usan:

- $\text{in}[u]$: posición de u en el arreglo lineal.
- $\text{out}[u]$: última posición dentro del subarbol de u .
- $\text{parent}[u]$: padre de u .
- $\text{depth}[u]$: profundidad.
- $\text{nxt}[u]$: cabeza de la cadena pesada donde está u .
- $\text{size}[u]$: tamaño de subarbol.

3.8 Propiedad de linealización

Si u está en una cadena pesada, entonces:

$$[\text{in}[\text{nxt}[u]], \text{in}[u]]$$

contiene todos los nodos desde la cabeza de la cadena hasta u . Como esos nodos son consecutivos en el arreglo, puedes preguntarle al segment tree por ese rango.

3.9 Consulta de camino con HLD

Para consultar el camino entre a y b :

1. Mientras $\text{nxt}[a] \neq \text{nxt}[b]$, los nodos están en cadenas distintas.
2. Tomas la cadena cuya cabeza está más profunda.
3. Consultas el rango desde la cabeza de esa cadena hasta el nodo.
4. Subes ese nodo al padre de la cabeza.
5. Cuando están en la misma cadena, haces una última query entre el nodo más alto y el más bajo.

La razón de escoger la cabeza más profunda es que esa cadena completa pertenece al camino $a \rightsquigarrow b$ y puedes eliminarla de golpe.

3.10 Correctitud de HLD path query

El algoritmo siempre mantiene que falta procesar el camino entre los nodos actuales a y b . Si sus cadenas son distintas, la cabeza más profunda no puede estar por encima del LCA de los nodos originales; por tanto, el segmento desde esa cabeza hasta el nodo actual pertenece completamente al camino. Después de consultarlo, subir al padre de la cabeza elimina exactamente esa parte. Al final ambos nodos están en la misma cadena, y el tramo restante es un solo intervalo.

3.11 Complejidad

Hay $O(\log n)$ cambios de cadena. Cada consulta al segment tree cuesta $O(\log n)$. Entonces:

$$\text{path query/update} = O(\log^2 n).$$

El preprocesamiento con DFS cuesta:

$$O(n).$$

3.12 Subtree queries con el mismo HLD

Como el DFS asigna tiempos consecutivos a un subarbol:

$$\text{subtree}(u) = [\text{in}[u], \text{out}[u]].$$

Por eso una query/update de subarbol es una sola operacion de segment tree:

$$O(\log n).$$

3.13 Nodos vs aristas

Si los valores estan en nodos, usas directamente $\text{in}[u]$. Si los valores estan en aristas, representas la arista $(\text{parent}[u], u)$ en la posicion de u . Entonces una query de camino debe tener cuidado de excluir la posicion del LCA si estas consultando aristas.

3.14 Cuando usar HLD

Usa HLD cuando:

- el arbol es fijo;
- hay muchas consultas entre pares de nodos;
- las consultas son sobre caminos;
- necesitas combinar con segment tree;
- aparecen frases como “path query”, “path update”, “max edge on path”, “sum on path”.

3.15 Cuando NO usar HLD

No es la primera opcion si:

- el arbol cambia con link/cut;
- solo necesitas LCA simple;
- solo preguntas subarboles, donde Euler timestamps bastan;
- el problema es conectividad general con inserciones y borrados de aristas.

4 Small-to-large trick / DSU on tree

4.1 Que problema resuelve

Small-to-large, tambien llamado DSU on tree, sirve para calcular informacion de cada subarbol de un arbol estatico. Es util cuando para cada nodo u quieres responder algo sobre todos los vertices de su subarbol.

Ejemplos:

- color mas frecuente en el subarbol de cada nodo;
- cantidad de colores distintos en cada subarbol;
- queries tipo “Tree and Queries”;
- “Lomsat gelral”;
- informacion que se puede mantener con operaciones add/remove.

4.2 Idea intuitiva

Procesar desde cero cada subarbol seria $O(n^2)$. Small-to-large evita eso:

- procesa hijos livianos y borra su informacion;
- procesa el hijo pesado y conserva su informacion;
- agrega encima los hijos livianos y el nodo actual;
- responde la query del nodo actual;
- si el padre no quiere conservar esta informacion, la borra.

4.3 Relacion con HLD

La misma idea heavy/light garantiza que un nodo solo se reinsertara por cada arista liviana en su camino hacia la raiz. Como hay $O(\log n)$ aristas livianas, cada nodo se agrega/elimina $O(\log n)$ veces. Por eso el total es $O(n \log n)$ operaciones de informacion.

4.4 Pseudocodigo teorico

1. Calcula tamaños de subarbol y el hijo pesado de cada nodo.
2. Para cada hijo liviano v , resuelve v con **keep=false**.
3. Resuelve el hijo pesado con **keep=true**.
4. Agrega al contenedor la informacion de todos los hijos livianos y de u .
5. Responde la pregunta de u con el contenedor actual.
6. Si **keep=false**, elimina la informacion del subarbol de u .

4.5 Correctitud

Cuando respondes el nodo u , el contenedor tiene exactamente los nodos del subarbol de u : el hijo pesado quedo conservado, los hijos livianos fueron reagregados y u fue agregado. Por tanto, cualquier funcion que dependa solo del multiconjunto de valores del subarbol se responde correctamente.

4.6 Complejidad

Si **add** y **remove** cuestan $O(1)$ o $O(\log n)$, el total es:

$$O(n \log n) \quad \text{o} \quad O(n \log^2 n)$$

segun el costo del contenedor.

5 Dynamic tree problem

5.1 Modelo

El dynamic tree problem mantiene un bosque de arboles enraizados bajo:

- $\text{link}(v, w)$: conecta dos arboles agregando una arista.
- $\text{cut}(v)$ o $\text{cut}(u, v)$: elimina una arista.
- $\text{findRoot}(v)$: encuentra la raiz/componente de v .
- path aggregates: suma, minimo o maximo en un camino.

La meta de los slides:

$O(\log n)$ amortizado por operacion.

5.2 Diferencia con HLD

HLD funciona cuando el arbol no cambia. Dynamic trees permiten cambiar la forma del bosque. Por eso HLD ya no basta como estructura directa. Link-cut trees usan ideas parecidas a HLD, pero las cadenas preferidas cambian segun las operaciones.

5.3 Operaciones tipicas

Operacion	Significado	Meta
$\text{access}(v)$	expone el camino raiz- v	$O(\log n)$ amort.
$\text{link}(v, w)$	conecta dos arboles	$O(\log n)$ amort.
$\text{cut}(v)$	corta una arista hacia el padre	$O(\log n)$ amort.
$\text{findRoot}(v)$	raiz del arbol de v	$O(\log n)$ amort.
$\text{pathQuery}(u, v)$	agregado en camino	$O(\log n)$ amort.

6 Link-cut trees

6.1 Idea central

Un link-cut tree representa cada arbol original mediante una coleccion de caminos preferidos. Cada camino preferido se guarda en un splay tree auxiliar. La estructura cambia los caminos preferidos con `access(v)`.

6.2 Preferred child

Para un nodo v , su preferred child es el hijo c tal que el ultimo acceso dentro del subarbol de v ocurrio en el subarbol de c .

6.3 Preferred edge y preferred path

La arista hacia el preferred child es preferred edge. Una preferred path es una cadena maxima de preferred edges. Las preferred paths son disjuntas por vertices.

6.4 Auxiliary tree

Cada preferred path se guarda en un splay tree:

- la llave es la profundidad en el arbol original;
- el subarbol izquierdo contiene ancestros;
- el subarbol derecho contiene descendientes;
- cada splay puede guardar agregados del camino.

6.5 Path-parent

Si una preferred path termina y existe una arista normal hacia arriba, el splay que representa la path guarda un puntero conceptual llamado path-parent hacia la siguiente path superior. Esto permite subir entre caminos.

6.6 La operacion `access(v)`

`access(v)` fuerza que el camino desde la raiz representada hasta v se convierta en una sola preferred path. Los slides la describen asi:

1. Hacer splay de v para llevarlo a la raiz de su auxiliary tree.
2. Cortar el hijo derecho de v : esos descendientes dejan de ser preferred.
3. Subir mediante path-parent al siguiente camino superior.
4. Hacer splay de ese nodo superior y pegar el camino anterior como hijo derecho.
5. Repetir hasta llegar a la raiz representada.

Efecto: despues de `access(v)`, v queda en la raiz de un splay que contiene exactamente el camino desde la raiz del arbol hasta v .

6.7 Por que `access` permite path queries

Si puedes exponer un camino como un splay, entonces el agregado del camino esta guardado en la raiz del splay. Para consultas entre dos nodos en una version enraizable se usa normalmente:

- `makeroot(u)`;
- `access(v)`;
- leer el agregado del splay de v .

La guia de slides enfatiza la idea conceptual de `access`; en una implementacion completa tambien aparecen lazy reversals para `makeroot`.

6.8 Analisis con HLD

Los slides usan HLD como herramienta de analisis, no necesariamente como parte de la implementacion. El punto es:

- Cambios al acceder a hijos livianos son pocos en cualquier camino: $O(\log n)$.
- Cambios asociados a hijos pesados pueden ser muchos individualmente, pero se pagan con una funcion potencial.
- El costo total de **access** queda $O(\log n)$ amortizado.

6.9 Correctitud conceptual

La estructura nunca cambia el arbol representado cuando solo hace **access**. Solo cambia como lo descompone en caminos preferidos. Como cada preferred path se guarda ordenada por profundidad, las rotaciones del splay preservan el orden del camino. Al cortar y pegar hijos derechos se actualiza exactamente que parte del camino raiz- v debe volverse preferred.

6.10 Cuando usar link-cut tree

Usa link-cut tree cuando:

- el grafo siempre es bosque;
- hay operaciones link y cut online;
- necesitas agregados en caminos;
- el arbol cambia, asi que HLD estatico no sirve.

6.11 Cuando NO usar link-cut tree

No lo uses como primera opcion si:

- el problema es offline: rollback DSU suele ser mas facil;
- solo hay inserciones: DSU basta;
- solo hay arbol estatico: HLD basta;
- solo quieres subtree aggregates: Euler-tour trees suelen ser mas naturales.

7 Euler-tour trees

7.1 Idea central

Un Euler-tour tree representa un arbol como una secuencia de visitas de un tour. Esa secuencia se guarda en un BST balanceado, por ejemplo treap, AVL o splay. Como el BST permite **split** y **merge**, puedes cortar y pegar subsecuencias para simular link/cut.

7.2 Secuencia Euler

Para un arbol no dirigido, cada arista se recorre dos veces: bajando y subiendo. Los slides dan una secuencia conceptual:

$$1 \rightarrow 2 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 3 \rightarrow 1.$$

Lo importante no es memorizar exactamente esa convencion, sino entender que el tour codifica la frontera del arbol y que cada subarbol aparece como un bloque manipulable en la secuencia.

7.3 Representacion en BST balanceado

El BST no se ordena por labels de vertices. Se ordena por posicion implicita en la secuencia. Cada nodo del BST puede guardar:

- tamano del segmento;
- suma/min/max del segmento;
- punteros a ocurrencias de vertices;
- informacion de componente.

7.4 Cut en Euler-tour tree

Para cortar un subarbol o una arista:

1. Encuentras las ocurrencias relevantes en la secuencia.
2. Haces splits para separar:

Before, Middle, After.

3. El bloque que representa la parte cortada queda separado.
4. Concatenas lo que debe seguir junto.

Cada split/merge cuesta $O(\log n)$.

7.5 Link en Euler-tour tree

Para unir el arbol de u como hijo de v :

1. Cortas la secuencia en la posicion donde quieres insertar.
2. Insertas/spliceas la secuencia completa del arbol de u .
3. Agregas las ocurrencias que representan la nueva arista.
4. Haces merge de todo.

7.6 Conectividad con Euler-tour trees

Dos vertices estan conectados si sus ocurrencias pertenecen al mismo BST raiz. Por eso una query **connected**(u,v) se reduce a comparar representantes del BST que contiene la ocurrencia de u y la ocurrencia de v .

7.7 Comparacion con link-cut trees

Criterio	Link-cut tree	Euler-tour tree
Mejor para	agregados en camino	agregados de subarbol/componente
Representacion	preferred paths en splay	tour en BST balanceado
Link/cut forest	si	si
Path aggregate	natural	menos directo
Subtree aggregate	menos natural	natural
Implementacion	compleja	mas secuencial

8 Dynamic connectivity en grafos generales

8.1 Problema

Mantener un grafo no dirigido bajo:

- `insert(u,v)`;
- `delete(u,v)`;
- `connected(u,v)`.

8.2 Por que es mas dificil que bosques

En un bosque, borrar una arista siempre parte un arbol en dos componentes. En un grafo general, puede haber otro camino alternativo. Entonces borrar una arista puede no cambiar nada. Si la arista borrada era parte del spanning forest, debes buscar una arista de reemplazo entre las dos nuevas partes.

8.3 Estrategia de los slides

Los slides resumen una solucion con:

- mantener spanning forests del grafo;
- particionar aristas en $\log n$ niveles;
- usar Euler-tour trees para mantener el bosque de cada nivel;
- costo amortizado $O(\log^2 n)$ en updates y $O(\log n)$ en queries.

8.4 Idea de niveles

Las aristas se organizan por niveles para controlar cuantas veces puede bajar o subir una arista antes de pagar su costo. Cuando se borra una tree edge, se buscan reemplazos entre niveles. La estructura por niveles evita revisar todas las aristas cada vez.

8.5 Lower bounds

Los slides mencionan resultados de Patrascu y Demaine: existe un trade-off inherente entre update time y query time. La moraleja para examen: no esperes una estructura trivial con update y query ambos constantes para fully dynamic connectivity general.

9 Rollback DSU y conectividad dinamica offline

9.1 Por que incluirlo

Aunque los slides hablan de Euler-tour trees para conectividad dinamica online, muchos problemas tipo Yosupo/Codeforces permiten leer todas las operaciones antes de responder. En ese caso, la herramienta mas importante es rollback DSU con segment tree sobre el tiempo.

9.2 DSU normal

DSU mantiene componentes bajo uniones:

- `find(x)` devuelve el representante.
- `union(a,b)` une las componentes.

Sirve perfecto para inserciones, pero no soporta borrados directamente.

9.3 Rollback DSU

Rollback DSU guarda un stack de cambios. Cada union registra que representante cambio, que tamano cambio y que agregado cambio. Luego puedes deshacer hasta un snapshot anterior.

Importante: para rollback normalmente no se usa path compression, porque seria dificil deshacer todos los cambios. Se usa union by size/rank, con find $O(\log n)$ o casi constante por altura controlada.

9.4 Segment tree sobre el tiempo

Cada arista tiene intervalos de vida: desde que se inserta hasta que se borra. Si una arista esta activa en el intervalo $[l, r]$, la agregas a los nodos del segment tree de tiempo que cubren ese intervalo.

Luego haces DFS sobre el segment tree:

1. Tomas snapshot del DSU.
2. Agregas todas las aristas guardadas en el nodo actual.
3. Si estas en una hoja de tiempo, respondes las queries de ese instante.
4. Si no, bajas a hijos.
5. Al salir, haces rollback al snapshot.

9.5 Correctitud

Cuando estas en una hoja t , el camino desde la raiz del segment tree hasta esa hoja contiene exactamente los nodos cuyos intervalos cubren t . Por tanto, el DSU contiene exactamente las aristas activas en el tiempo t . Las respuestas de conectividad o agregado por componente son correctas. El rollback restaura el estado para explorar otro intervalo sin contaminarlo con aristas que no estan activas ahi.

9.6 Complejidad

Cada intervalo de vida de una arista se guarda en $O(\log q)$ nodos del segment tree. Si hay m intervalos:

$$O(m \log q)$$

uniones. Cada union/find cuesta $O(\log n)$ sin path compression, o $O(1)$ amortizado si se analiza por union by size en la practica teorica segun la variante. Una cota segura:

$$O((m \log q + q) \log n).$$

Espacio:

$$O(m \log q + n).$$

9.7 Component sum con vertex add

En problemas como “Dynamic Graph Vertex Add Component Sum”, además de aristas activas hay updates de valor en vértices y queries de suma de componente. La idea teórica:

- DSU guarda $sum[root]$ de cada componente.
- Al unir componentes, suma sus valores.
- Para vertex add offline, puedes tratar el cambio de valor como evento en la hoja de tiempo o usar técnicas de contribución por intervalo según el formato.
- En una query, respondes $sum[find(v)]$.

Si los vertex updates son online mezclados con rollback, debes registrar en el stack también cambios de suma para poder deshacerlos.

9.8 Cuando usar rollback DSU

Usalo cuando:

- hay inserciones y borrados de aristas;
- puedes leer todas las operaciones antes de responder;
- las queries son conectividad o agregado por componente;
- el problema no exige respuesta online real.

10 Imperial Roads y max edge on path

10.1 Conexion con dynamic/trees

El warm-up de los slides pregunta por Imperial Roads: si el rey quiere forzar una carretera en el MST, como cambia la solucion. La idea clave usa propiedades de MST y consultas de maximo en camino.

10.2 Propiedad del ciclo

Toma un MST T . Si agregas una arista no-arbol $e = (u, v)$ de peso w , se forma un ciclo: la arista nueva mas el camino entre u y v en T .

Para obtener el mejor MST que incluye e , debes quitar del ciclo la arista de mayor peso dentro del camino de u a v en T . Si ese maximo pesa M , el nuevo costo es:

$$\text{cost}(T) + w - M.$$

10.3 Estructura

Necesitas responder max edge on path en el MST. Puedes usar:

- LCA con binary lifting guardando maximo por salto;
- HLD + segment tree;
- RMQ/LCA si el problema se formula asi.

10.4 Correctitud

Al agregar e , cualquier arbol que incluya e y mantenga conectividad debe eliminar una arista del ciclo. Para minimizar el peso total, conviene eliminar la arista mas pesada del ciclo que no sea obligatoria. Como e se fuerza, se elimina la maxima del camino original. Esta es la propiedad de ciclo del MST.

11 Como reconocer que estructura usar

Frase del problema	Usa	Por que
Solo insertan aristas y preguntan connected	DSU	no hay borrados
Insert/delete/connected offline	rollback DSU + segment tree	conoces intervalos de vida tiempo
Forest con link/cut online	link-cut tree o Euler-tour tree	cambia la forma del bosque
Path max/sum en arbol fijo	HLD + segment tree	camino se parte en pocos intervalos
Subtree add/query en arbol fijo	Euler timestamps + segment tree	subarbol es intervalo
Subtree informacion por cada nodo	DSU on tree	conserva hijo pesado
Path aggregate en arbol dinamico	link-cut tree	expone camino con access
Component aggregate en bosque dinamico	Euler-tour tree	componente como tour/secuencia
Forzar arista en MST	MST + max edge path	propiedad del ciclo

12 Plantillas de solucion teorica

12.1 Plantilla HLD

1. Enraizo el arbol y calculo size, *parent*, depth.
2. Marco como pesado el hijo de mayor subarbol.
3. Linealizo cadenas pesadas con in y *next*.
4. Construyo segment tree sobre el arreglo lineal.
5. Para una query de camino, mientras las cabezas difieran, proceso la cadena de cabeza mas profunda y subo.
6. Correctitud: cada segmento procesado pertenece al camino y al final queda un solo segmento en la misma cadena.
7. Complejidad: $O(n)$ preproceso, $O(\log^2 n)$ por path query/update.

12.2 Plantilla link-cut tree

1. Represento cada preferred path con un splay auxiliar.
2. Mantengo preferred child segun el ultimo access.
3. `access(v)` convierte el camino raiz- v en una preferred path.
4. Link/cut se implementan exponiendo caminos y modificando punteros.
5. Los agregados de camino se guardan en las raices de los splays.
6. Correctitud: access cambia solo la representacion, no el arbol representado.
7. Complejidad: $O(\log n)$ amortizado por operacion.

12.3 Plantilla Euler-tour tree

1. Represento cada arbol por una secuencia Euler guardada en BST balanceado.
2. Link se vuelve cortar una posicion y splicear otra secuencia.
3. Cut se vuelve separar un bloque con split y recombinar con merge.
4. Connected compara si dos ocurrencias pertenecen al mismo BST.
5. Correctitud: el tour codifica exactamente la componente arbol.
6. Complejidad: $O(\log n)$ por split/merge, por tanto $O(\log n)$ por operacion de bosque.

12.4 Plantilla rollback DSU

1. Leo todas las operaciones.
2. Convierto cada arista en intervalos de tiempo donde esta activa.
3. Inserto cada intervalo en un segment tree de tiempo.
4. Recorro el segment tree con DFS y rollback DSU.
5. En cada hoja respondo con el DSU actual.
6. Correctitud: en la hoja t estan unidas exactamente las aristas activas en t .
7. Complejidad: $O(m \log q)$ uniones, cada una con costo del DSU.

13 Mapa de slides cubiertos

CS3014____Heavy_Light_Decomposition.pdf

- Pagina 1: motivacion de consultas/modificaciones en caminos y definicion general de HLD.
- Pagina 2: dos definiciones de arista pesada y prueba de que una arista liviana reduce el subarbol a la mitad.
- Pagina 3: implementacion base, arreglos **in**, **out**, **nxt**, profundidad y propiedad del rango de una cadena.
- Pagina 4: path query, path update, subtree update y nota para valores en aristas.
- Pagina 5: extension a small-to-large trick y pseudocodigo conceptual.
- Pagina 6: implementacion de add/solve y problemas de practica.

eda_slides_08_dynamic_graph.pdf

- Paginas 1–4: overview y warm-up con Imperial Roads.
- Paginas 5–6: dynamic tree problem, link/cut, findRoot y path aggregates.
- Paginas 7–10: preferred paths, auxiliary trees y operacion access.
- Paginas 11–14: HLD como herramienta de analisis amortizado.
- Paginas 15–19: resumen, operaciones y referencias.

eda_slides_09_dynamic_graph.pdf

- Paginas 1–5: dynamic connectivity, fully dynamic, incremental y decremental.
- Pagina 6: contraste link-cut trees vs Euler-tour trees.
- Paginas 7–9: Euler-tour tree mechanics, link y cut con split/merge.
- Pagina 10: fully dynamic connectivity en grafos generales con niveles y Euler-tour trees.
- Pagina 11: resumen y lower bounds de Patrascu–Demaine.
- Paginas 12–14: cierre, referencias y acknowledgements.

14 Apéndice: mapa página por página de los slides dinámicos

Este apéndice lista cada página/slide de la carpeta fuente y su contenido principal. Algunas páginas son portadas, separadores o cierres; se conservan para trazabilidad.

CS3014_____Heavy_Light_Decomposition.pdf (6 páginas)

Pag. 1

CS3014 - Estructuras de Datos Avanzadas Notas de clase 01 Dynamic Graphs - Heavy-light Decomposition 8 de junio de 2026 Prof. Víctor Racs' o Galv' an Oyola 1. Descomposiciones de ' arboles Hasta ahora hemos tenido algunas ideas para poder realizar consultas espec' ificas (Euler Tour para LCA) o consultas/modificaciones en sub' arboles (DFS Timestamps) de manera eficiente. Sin embargo, tambi' en es posible recibir consultas/modificaciones sobre las aristas/nodos del camino entre un par de nodos. Para ello, podemos...

Pag. 2

CS3014 - Estructuras de Datos Avanzadas Notas de clase 01 respectivas. 1.1.1. Mayor que la mitad Arista pesada: Aquella arista (u, v) tal que el tama~ no del sub' arbol debes mayor que la mitad el tama~ no del sub' arbol deu. $(2\text{subtree}[v] > \text{subtree}[u])$. 1.1.2. Mayor que sus hermanos Arista pesada: Aquella arista (u, v) tal que el tama~ no del sub' arbol debes mayor que los tama~ nos de los sub' arboles de sus hermanosw, si hay alg' un empate, se considera solo una como pesada. $(\text{subtree}[v] \geq \text{subtree}[w], w \text{ in children}[u])$...

Pag. 3

CS3014 - Estructuras de Datos Avanzadas Notas de clase 01 8} 9int $v = G[u][i]$; 10if($v == p$) continue; 11DFS(v, u); 12if($\text{subtree}[v] > \text{subtree}[G[u][0]]$){ 13swap($G[u][i], G[u][0]$); 14} 15 $\text{subtree}[u] += \text{subtree}[v]$; 16} 17if($p \neq -1$){ 18 $G[u].\text{pop_back}()$; 19} 20} 21 22void HLD(int u){ 23in[u] = $T++$; 24for(int $v : G[u]$){ 25par[v] = u ; 26nxt[v] = ($v == G[u][0] ? \text{nxt}[u] : v$); 27h[v] = h[u] + 1; 28HLD(v); 29} 30out[u] = $T - 1$; 31} Notemos que estaremos guardando algunos arreglos extra: in y outson los timestamps de entrada y sal...

Pag. 4

CS3014 - Estructuras de Datos Avanzadas Notas de clase 01 lint path(int a, b){ 2int $res = -2e9$; 3while($\text{nxt}[a] \neq \text{nxt}[b]$){ 4if(h[$\text{nxt}[a]$] < h[$\text{nxt}[b]$]) swap(a, b); 5 $res = \max(res, \text{query}(\text{in}[\text{nxt}[a]], \text{in}[a]))$; 6a = par[$\text{nxt}[a]$]; 7} 8if(h[a] > h[b]) swap(a, b); 9 $res = \max(res, \text{query}(\text{in}[a], \text{in}[b]))$; 10return res ; 11} Es sencillo notar que el bucle se ejecutar' a hasta que est' en ambos nodos en la misma cadena, para lo cual realizamos una consulta extra para cubrir dicho rango de nodos. La complejidad de este algoritmo ...

Pag. 5

CS3014 - Estructuras de Datos Avanzadas Notas de clase 01 Consideremos ahora el caso en el que se nos da un ' arbolT, sobre el cual nosotros queremos responder a cierta informaci' on de cada uno de sus nodosu, considerando que dicha informaci' on depende del sub' arbol deu. Es importante recordar la definici' on de arista pesada y ligera, pues debido a esta se da que existen una cantidad de $O(\log n)$ aristas ligeras en el camino desde la ra' iz del ' arbol hasta cualquier nodo u . Entonces, si pudi' eramos procesar cad...

Pag. 6

CS3014 - Estructuras de Datos Avanzadas Notas de clase 01 7int $v = G[u][i]$; 8if($v == p$) continue; 9DFS(v, u); 10if($\text{subtree}[v] > \text{subtree}[G[u][0]]$){ 11swap($G[u][i], G[u][0]$); 12} 13 $\text{subtree}[u] += \text{subtree}[v]$; 14} 15if($p \neq -1$){ 16 $G[u].\text{pop_back}()$; 17} 18} 19 20void add(int $u, \text{int val}, \text{int start}$){ 21if($\text{val} == 1$) agregarInfo(u); 22else quitarInfo(u); 23for(int $i = \text{start}; i < G[u].\text{size}(); i++)$ { 24add($G[u][i], \text{val}, 0$); // Solo debemos restringir en el primer nivel 25} 26} 27 28void solve(int $u, \text{int keep}$){ 29for(int $i = 1; i \dots$

eda_slides_08_dynamic_graph.pdf (19 páginas)

Pag. 1

Dynamic Graph CS3014 - Estructura de Datos Avanzados Luciano A. Romero Calla lromeroc@utec.edu.pe 2026-1

Pag. 2

Overview 1. Warming Up 2. The Dynamic Tree Problem 3. Structural Tool: Preferred Paths 4. Heavy-Light Decomposition (HLD) 5. Summary 2 / 16

Pag. 3

Warming Up 1. Warming Up 3 / 16

Pag. 4

Warming Up Warming Up Problem Remember the problem of Imperial Roads. Now the king may want to keep a chosen road. How does this change your previous solution? A B C D E F 50 70 80 95 20 30 40 50 60
4 / 16

Pag. 5

The Dynamic T ree Problem 2. The Dynamic T ree Problem 5 / 16

Pag. 6

The Dynamic T ree Problem The Dynamic T ree Problem The Challenge: Maintain a forest of rooted trees under: - Structural updates (Link/Cut). - Connectivity queries (FindRoot). - Path aggregates (Min/Max/Sum). Efficiency Goal $O(\log n)$ amortized time per operation. r v w link(v, w)? 6 / 16

Pag. 7

Structural T ool: Preferred Paths 3. Structural T ool: Preferred Paths 3.1 Internal Representation: Auxiliary Trees 3.2 The Foundation: access(v) 7 / 16

Pag. 8

Structural T ool: Preferred Paths Structural T ool: Preferred Paths We decompose the tree into vertex-disjoint paths based on access patterns. - Preferred Child: The child of v such that the last access inv's subtree was inc's subtree. - Preferred Edge: Connection to preferred child. - Preferred Path: Maximal chain of preferred edges. 1 2 3 4 5 6 Preferred Path Normal Edge 8 / 16

Pag. 9

Structural T ool: Preferred Paths Internal Representation: Auxiliary T rees Internal Representation: Auxiliary T rees Each preferred path is stored in an Auxiliary Tree (Splay Tree). - Order: Keyed by depth in the original tree. - Left Subtree: Ancestors (smaller depth). - Right Subtree: Descendants (larger depth). 2 1 4 Splay for Path{1,2,4} 3 6 path-parent 9 / 16

Pag. 10

Structural T ool: Preferred Paths The Foundation: access(v) The Foundation: access(v) This operation forces the root-to- v path to become a single preferred path. 1. Splay v to the root of its Aux-Tree. 2. Discard v's right child (it becomes a separate path). 3. Climb via path-parent to w = pp(v). 4. Splay w and make it its new right child. 5. Repeat until the represented root is reached. Effect After access(v), node v is at the root of a Splay tree containing exactly the path from v to the tree root. 10 / 16

Pag. 11

Heavy-Light Decomposition (HLD) 4. Heavy-Light Decomposition (HLD) 4.1 HLD: Why the Logarithmic Bound? 4.2 Complexity Proof Sketch 11 / 16

Pag. 12

Heavy-Light Decomposition (HLD) Heavy-Light Decomposition (HLD) To prove $O(\log n)$, we use HLD as an analysis tool (not part of the implementation). Definition Let $\text{size}(v)$ be the number of nodes in v's subtree. - Edge(u,v) is Heavy if $\text{size}(v) > \frac{1}{2} \text{size}(u)$. - Edge(u,v) is Light otherwise. Crucial Lemma: Any path from root to node contains at most $\log_2 n$ light edges. 12 / 16

Pag. 13

Heavy-Light Decomposition (HLD) HLD: Why the Logarithmic Bound? HLD: Why the Logarithmic Bound? Moving down a Light Edge reduces the subtree size by at least half. 16 9 6 4 4 5 H L ($6 \leq 16/2$) Max Light Edges $\sim \log_2(16) = 4$ 13 / 16

Pag. 14

Heavy-Light Decomposition (HLD) Complexity Proof Sketch Complexity Proof Sketch The cost of access(v) is the number of preferred- edge changes. Preferred-Child Changes - Light Changes: Accessing a light child. Only $\log n$ exists on any path. - Heavy Changes: Accessing a heavy child. Can be many, but they create light edges for future operations. Total Cost By potential function analysis (based on HLD), the number of heavy-to-light transitions is bounded. Total: $O(\log n)$ amortized. 14 / 16

Pag. 15

Summary 5. Summary 15 / 16

Pag. 16

Summary Summary - Link-Cut Treessolve dynamic forest problems in $O(\log n)$. - Key Subroutine: `access(v)` handles the path logic. - Analysis: Heavy-Light Decomposition provides the amortized guarantee. Operation Mechanism Complexity `Access(v)` Climbing Aux Trees $O(\log n)$ `Link(v, w)` Access + pp link $O(\log n)$ `Cut(v)` Access + remove left $O(\log n)$ `PathSum(v)` Access + Splay Aggregate $O(\log n)$ 16 / 16

Pag. 17

Thanks

Pag. 18

References References 18 / 16

Pag. 19

Acknowledgements Acknowledgements The course slides are based on the lectures from previous editions and on similar lectures elsewhere. List of credits: Erick Demaine, Keith Schwarz 19 / 16

eda_slides_09_dynamic_graph.pdf (14 paginas)

Pag. 1

Dynamic Graph CS3014 - Estructura de Datos Avanzados Luciano A. Romero Calla lromeroc@utec.edu.pe 2026-1

Pag. 2

Overview 1. Dynamic Connectivity 2. Euler-Tour Trees 2 / 11

Pag. 3

Dynamic Connectivity 1. Dynamic Connectivity 1.1 Problem Categorization 1.2 Tree-Structured Solutions: A Contrast 3 / 11

Pag. 4

Dynamic Connectivity Dynamic Connectivity Goal Maintain an undirected graph subject to updates and queries. Operations - `insert(u, v)`: Add edge (u, v) . - `delete(u, v)`: Remove edge (u, v) . - `connected(u, v)`: Are u and v in the same connected component? $u \neq v$ `insert(u, v)` 4 / 11

Pag. 5

Dynamic Connectivity Problem Categorization Problem Categorization Fully Dynamic Admits arbitrary interleavings of edge insertions and deletions. Incremental / Decremental Partially dynamic variants restriction to only insertions or only deletions. 5 / 11

Pag. 6

Dynamic Connectivity Tree-Structured Solutions: A Contrast Tree-Structured Solutions: A Contrast Before considering general graphs, we resolve dynamic connectivity within forests. Metric / Feature Link-Cut Trees (Sleator-Tarjan) Euler-Tour Trees (Henzinger-King) Primary Layout Heavy-Light Decomposition Flattened Traversal (Euler Tour) Aggregate Bounds Path Aggregates ($O(\log n)$) Subtree Aggregates ($O(\log n)$) Implementation Complex splay-tree interlinking Single balanced BST representation Edge Queries Difficult to locate...

Pag. 7

Euler-Tour Trees 2. Euler-Tour Trees 7 / 11

Pag. 8

Euler-Tour Trees Euler-Tour Tree Mechanics Concept: Convert a tree T into a sequential format by tracking its traversal boundary. Every undirected edge is spanned exactly twice (downward and upward). - Each node visit maps to an index in a sequence. - Sequence backed by a balanced BST (e.g., Treap, AVL). - Keys are implicitly defined by sequential array order. 1 2 3 Sequence: 1->2->2->1->3->3->1 8 / 11

Pag. 9

Euler-Tour Trees Structural Mutations: Link & Cut By tracking pointers to the first and last occurrence of node v , we manipulate subtrees in $O(\log n)$ time via basic BST splits and concatenations. `Cut(v)` Removes the

subtree rooted at v . 1. Identify v 's first and last visit in the BST. 2. Split sequence into three: Before- v , Subtree- v , After- v . 3. Concatenate Before- v and After- v . Link(u, v) Attaches u 's tree as a child of v . 1. Split v 's sequence right before its final occurrence. 2. Splice the entire sequence of u into the gap. 3. Merge...

Pag. 10

Euler-Tour Trees Fully Dynamic Connectivity in General Graphs How do we handle general graphs rather than isolated trees? - Maintain spanning forests of the graph. - Partition edges into $\log n$ levels based on assigned "weights" or frequencies. - Use an Euler-Tour tree to maintain the spanning forest of each level. - Amortized Cost: $O(\log^2 n)$ for updates, $O(\log n)$ for queries. Level Forest (BST) Level $i+1$ Forest edge push 10 / 11

Pag. 11

Euler-Tour Trees Summary & Lower Bounds Key Takeaways: - Euler-tour trees are simpler to implement and analyze than Link-Cut trees for aggregate subtree information. - General graph dynamic connectivity can be achieved in $O(\log^2 n)$ time using level-partitioned Euler-Tour trees. Lower Bounds (Patrascu & Demaine): - $O(x \log n)$ update requires $(\log n / \log x)$ query time. - $O(x \log n)$ query requires $(\log n \log x)$ update time. - Conclusion: There is an inherent trade-off in dynamic graph structures! 11 / 11

Pag. 12

Thanks

Pag. 13

References References 13 / 11

Pag. 14

Acknowledgements Acknowledgements The course slides are based on the lectures from previous editions and on similar lectures elsewhere. List of credits: Erick Demaine, Keith Schwarz 14 / 11

Cierre: si tienes poco tiempo

Prioriza así:

1. HLD completo: heavy/light, lema de mitad, path query, path update, subtree interval.
2. DSU y rollback DSU offline para conectividad dinamica de problemas tipo Yosupo.
3. Link-cut trees a nivel conceptual: preferred paths, auxiliary splay, access, link/cut.
4. Euler-tour trees: secuencia, BST balanceado, split/merge, conectividad.
5. Small-to-large trick para queries por subarbol.
6. Imperial Roads: MST + max edge on path.

Si debes responder en papel y no implementar, lo mas importante es identificar el modelo correcto: arbol fijo, bosque dinamico, grafo general online u offline.